UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

EVALUACION	PARCIAL	**GRUPO**	M4A	**FECHA**	27/11/2025
MATERIA	DISEÑO DE APLICACIONES				
CARRERA	ANALISTA EN TECNOLOGIAS DE INFORMACIÓN / ANALISTA PROGRAMADOR				
CONDICIONES	– Puntos: 40 – Sin material – Duración 2 horas				

Como parte de su programa de incentivo al turismo, el gobierno de la provincia de Chubut decide implementar un sistema de venta anticipada de tickets para su ya centenario tren patagónico a vapor, mejor conocido como “La Trochita”.

Se realizó la puesta en valor de vías y de dos formaciones con diferentes configuraciones, cada una de las cuales realiza un viaje de recorrido fijo partiendo y regresando a la misma estación de partida a la totalidad de los pasajeros.

Cada viaje tiene formación asignada que admite diferentes configuraciones: incluye un coche comedor y de 1 a 3 coches de pasajeros. De cada coche se conoce su capacidad medida en cantidad de pasajeros sentados. Cada viaje tiene una tarifa asignada para cada tipo de coche, en otras palabras, la tarifa de un viaje cambia en función de la ubicación (tipo de coche comedor ó coche de pasajeros).

Existe un plan de descuentos en función a la edad y residencia del pasajero:

Tipo de Pasajero	**Descuento**
Residente Nacional	20%
Residente Extranjero	10% para los primeros 20 pasajes vendidos a extranjeros en la formación en cuestión (sin importar el tipo de coche/ubicación)
Residente Nacional Jubilado	50%
Menores de 12 años	50% sin importar su procedencia

Un viaje puede estar programado, en curso o finalizado. Los viajes programados pueden tener cambios en los pasajes, mientras los otros dos no.

Caso de Uso: Consulta de viajes por rango de fecha

Descripción

El caso de uso permite al usuario consultar los viajes de La Trochita filtrando por rango de fecha y conocer datos como estado del viaje, cantidad de pasajes vendidos y monto recaudado entre otros.

Flujo básico

Paso	**Usuario**	**Sistema**
1		Muestra en pantalla las opciones para filtrar por fecha desde y fecha hasta.
2	Ingresa fecha desde, fecha hasta e indica que desea realizar la consulta	Verifica el ingreso de la fecha desde y fecha hasta.

Paso Usuario	Sistema
3	Despliega la consulta mostrando en pantalla: a. Fecha desde y hasta especificadas por el usuario. b. Datos de cada viaje: i. Fecha ii. Estación de origen/destino iii. Estado del viaje iv. Cantidad total de pasajes vendidos c. Monto total de pasajes vendidos (incluyendo descuentos según corresponda).

<u>Postcondiciones</u>
Los viajes "programado" y "en curso" que se observan, deben dejar de observarse en caso de cerrar la vista.

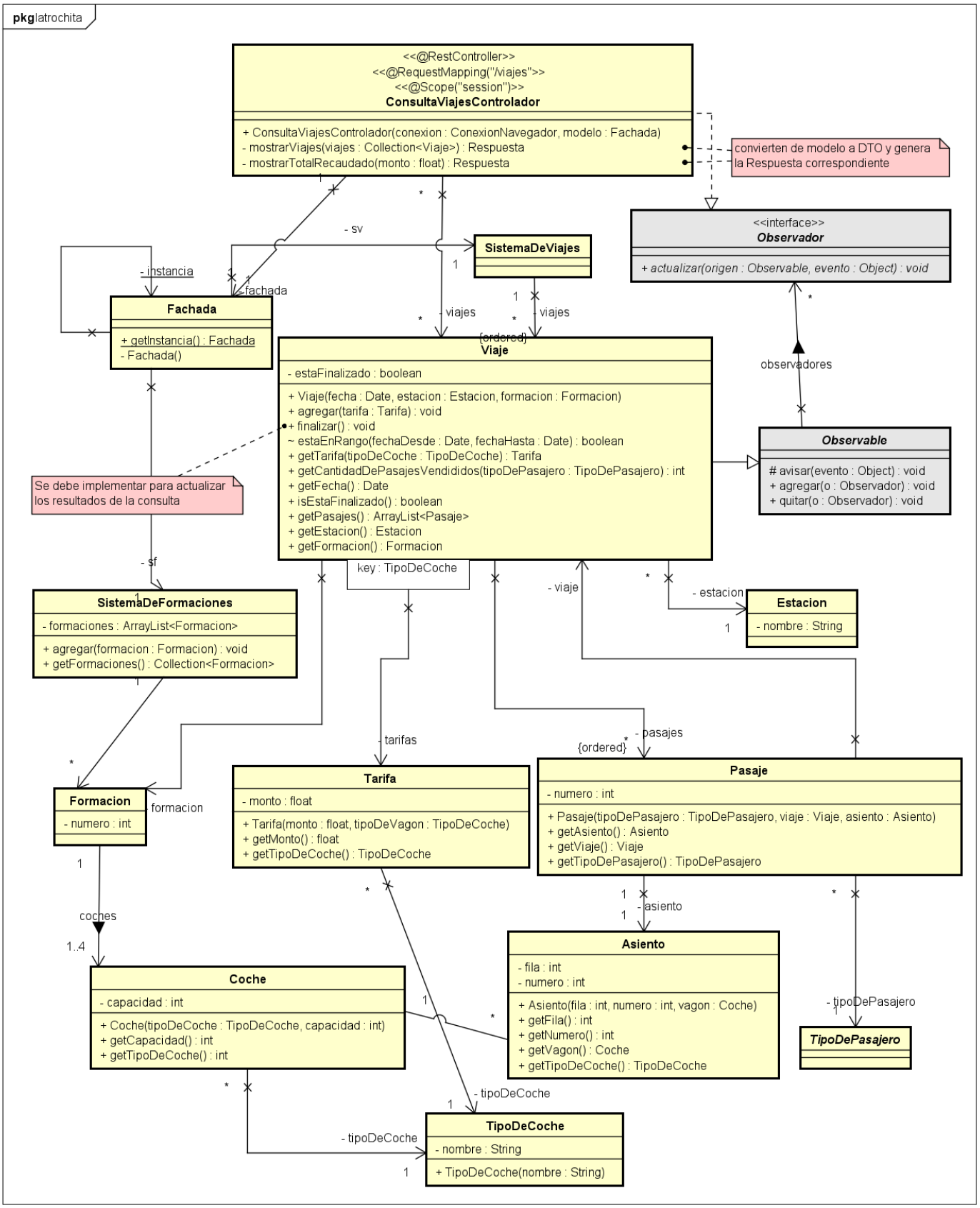
<u>Flujos alternativos</u>

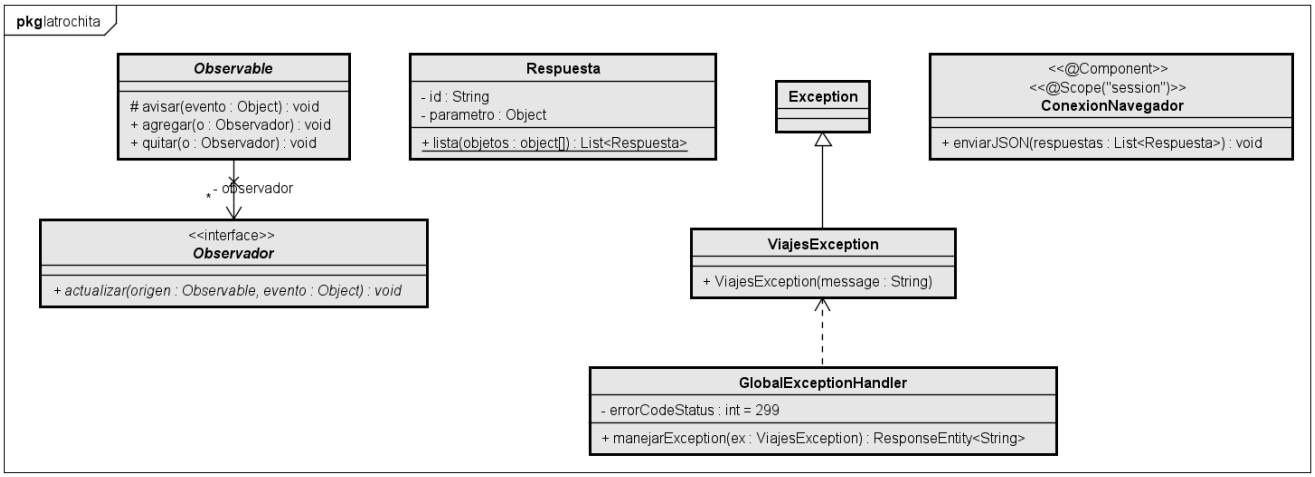
A2 – Falta al menos una de las fechas requeridas:
- El sistema:
  - Muestra mensaje:
    "Las fechas desde y hasta son requeridas"
  - Se retorna al paso 1 del curso normal.

A3 – No hay viajes registrados para el rango indicado:
- El sistema:
  - Muestra mensaje: "No se encontraron viajes registrados para el rango de fechas indicado"
  - Fin.

IMPORTANTE: Tenga en cuenta que este caso de uso puede ser ejecutado simultáneamente con otros casos de uso. Por lo tanto, debe **asegurarse que la solución del caso de uso contemple la actualización del resultado del reporte cuando un viaje que está dentro del rango de fechas cambia a finalizado. A tales efectos debe también <u>implementar el método finalizar de la clase Viaje</u>.**

El siguiente es el diagrama de diseño de la solución:


pkg latrochita
<<@RestController>>
<<@RequestMapping("/viajes">>
<<@Scope("session")>>
ConsultaViajesControlador
+ ConsultaViajesControlador(conexion : ConexionNavegador, modelo : Fachada)
- mostrarViajes(viajes : Collection<Viaje>) : Respuesta
- mostrarTotalRecaudado(monto : float) : Respuesta
convierten de modelo a DTO y genera la Respuesta correspondiente
<<interface>>
Observador
+ actualizar(origen : Observable, evento : Object) : void
- sv
SistemaDeViajes
- instancia
- fachada
Fachada
+ getInstancia() : Fachada
- Fachada()
- viajes
- viajes
{ordered}
Viaje
- estaFinalizado : boolean
+ Viaje(fecha : Date, estacion : Estacion, formacion : Formacion)
+ agregar(tarifa : Tarifa) : void
+ finalizar() : void
~ estaEnRango(fechaDesde : Date, fechaHasta : Date) : boolean
+ getTarifa(tipoDeCoche : TipoDeCoche) : Tarifa
+ getCantidadDePasajesVendididos(tipoDePasajero : TipoDePasajero) : int
+ getFecha() : Date
+ isEstaFinalizado() : boolean
+ getPasajes() : ArrayList<Pasaje>
+ getEstacion() : Estacion
+ getFormacion() : Formacion
key : TipoDeCoche
observadores
Observable
avisar(evento : Object) : void
+ agregar(o : Observador) : void
+ quitar(o : Observador) : void
Se debe implementar para actualizar los resultados de la consulta
- sf
SistemaDeFormaciones
- formaciones : ArrayList<Formacion>
+ agregar(formacion : Formacion) : void
+ getFormaciones() : Collection<Formacion>
- viaje
- estacion
Estacion
- nombre : String
- tarifas
- pasajes
{ordered}
Tarifa
- monto : float
+ Tarifa(monto : float, tipoDeVagon : TipoDeCoche)
+ getMonto() : float
+ getTipoDeCoche() : TipoDeCoche
Pasaje
- numero : int
+ Pasaje(tipoDePasajero : TipoDePasajero, viaje : Viaje, asiento : Asiento)
+ getAsiento() : Asiento
+ getViaje() : Viaje
+ getTipoDePasajero() : TipoDePasajero
Formacion
- numero : int
formacion
coches
1..4
- asiento
Asiento
- fila : int
- numero : int
+ Asiento(fila : int, numero : int, vagon : Coche)
+ getFila() : int
+ getNumero() : int
+ getVagon() : Coche
+ getTipoDeCoche() : TipoDeCoche
- tipoDePasajero
TipoDePasajero
Coche
- capacidad : int
+ Coche(tipoDeCoche : TipoDeCoche, capacidad : int)
+ getCapacidad() : int
+ getTipoDeCoche() : int
- tipoDeCoche
TipoDeCoche
- nombre : String
+ TipoDeCoche(nombre : String)
- tipoDeCoche



Requerimientos de diseño
Utilizar estrictamente el diagrama de clases dado en la letra como base para su solución. Solo debe agregar lo estrictamente necesario para la resolución del caso de uso.

Ítems	Puntos
Arquitectura, división lógica y MVC	13
Principio experto	7
Patrón Observador	7
Manejo de excepciones	3
Diseño polimórfico	7

Se pide:

A) Implementar en java el caso de uso descripto, exceptuando la vista, sí debe implementar el controlador necesario.

No es necesario implementar constructores ni getters/setters salvo que sean necesarios para comprensión de la solución. **Los métodos y atributos que aparecen en el diagrama ya se encuentran implementados.**

La implementación no debe contener lógica repetida.

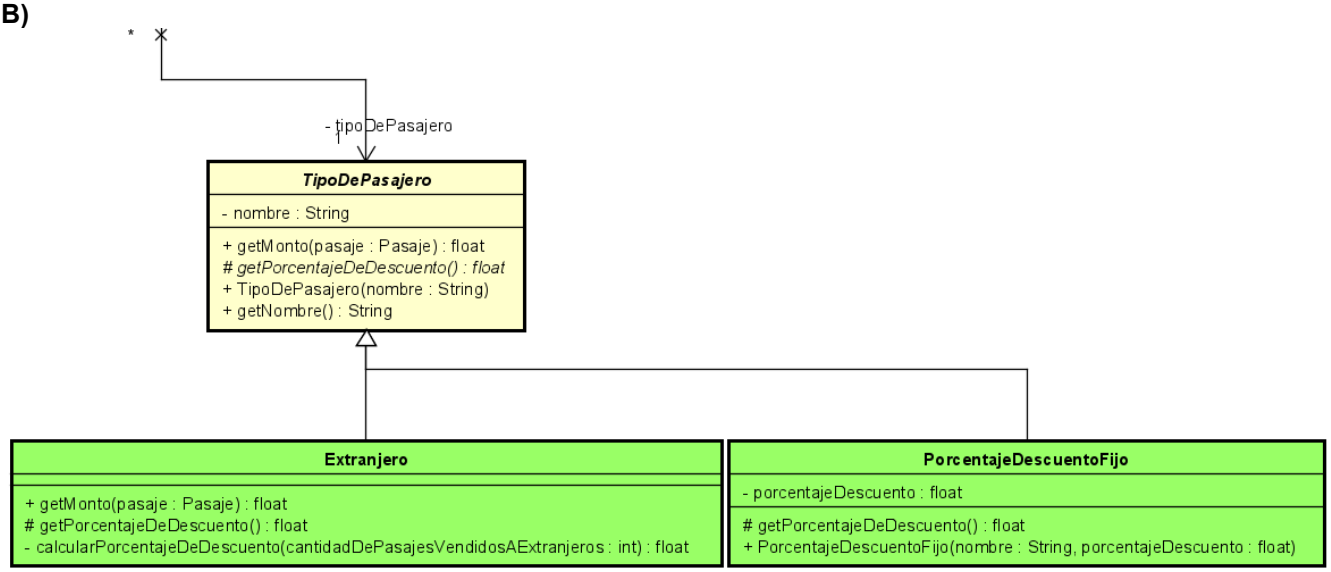
No es necesario implementar las clases Fachada y ViajesException. Tampoco Observable ni la interface Observador. Tampoco es necesario implementar otras clases que no utilice en su solución o que solo contengan métodos get y set. No debe implementar los DTO que necesite para su solución y tampoco los métodos privados que generan los objetos Respuesta indicados en el diagrama UML en la clase ConsultaViajesControlador.

B) Dibuje **las clases que haya agregado en su solución en notación UML** incluyendo las firmas de los métodos implementados. Si las clases están asociadas a alguna de las clases indicadas en la letra también debe incluirlas en el dibujo **(3 puntos)**.

SOLUCION PROPUESTA
A)

```
@RestController
@RequestMapping("/viajes")
@Scope("session")
public class ConsultaViajesControlador implements Observador {
   private final ConexionNavegador conexion;
   private final Fachada fachada;
   private Collection<Viaje> viajes;
   private Date fechaDesde, fechaHasta;
         @GetMapping("/buscar")
   public Lista<Respuesta> buscar(Date fechaDesde, Date fechaHasta) throws ViajesException{
         Collection<Viaje> resultado = fachada.getViajes(fechaDesde, fechaHasta);

                              List<Respuesta> respuestas;

         if (resultado.isEmpty()) {
            respuestas = Respuesta.lista(mostrarMensaje("No se encontraron viajes registrados para el rango de
fechas inidicado."));
         } else {
            desSuscribir(viajes);
            this.viajes = resultado;
            this.fechaDesde = fechaDesde;
            this.fechaHasta = fechaHasta;
            suscribir(viajes);
            respuestas = Respuesta.lista(viajemostrarViajes(viajes),

         mostrarTotalRecaudado(Viaje.calcularMontoTotalRecaudado(this.viajes)));
         }
                              return respuestas;
   }
         @PostMapping("/vistaCerrada")
   public void salir() {
                  desSuscribir(this.viajes);
   }
   @Override
   public void actualizar(Observable origen, Object evento) {
      if (evento == "VIAJE_FINALIZADO") {
         this.buscar(fechaDesde, fechaHasta);
      }
   }
   private void desSuscribir(Collection<Viaje> viajes) {
                  if(viajes!=null){
                              for (Viaje v : viajes) {
                                          v.quitar(this);
                              }
                  }
   }
   private void suscribir(Collection<Viaje> viajes) {
      for (Viaje v : viajes) {
         if(!viaje.isEstaFinalizado(){
                                          v.agregar(this);
                              }
      }
```

```
    }
}
public class SistemaDeViajes {
    Collection<Viaje> getViajes(Date fechaDesde, Date fechaHasta) throws ViajesException {
        if (fechaDesde == null || fechaHasta == null) {
            throw new ViajesException("Las fechas desde/hasta son requeridas.");
        }
        ArrayList<Viaje> resultado = new ArrayList<>();
        for (Viaje v : viajes) {
            if (v.estaEnRango(fechaDesde, fechaHasta)) {
                resultado.add(v);
            }
        }
        return resultado;
    }
}
public class Viaje extends Observable {
    private Date fecha;
    static float calcularMontoTotalRecaudado(Collection<Viaje> viajes) {
        float monto = 0f;
        for (Viaje v : viajes) {
            monto = +v.getMontoTotal();
        }
        return monto;
    }
    public void finalizar() {
        if (!estaFinalizado) {
            avisar("VIAJE_FINALIZADO");
            estaFinalizado = true;
        }
    }
    float getMontoTotal() {
        float total = 0f;
        for (Pasaje p : this.pasajes) {
            total = +p.getMonto();
        }
        return total;
    }
}
public class Pasaje {
    public float getMonto() {
        return tipoDePasajero.getMonto(this);
    }
}
public abstract class TipoDePasajero {
    public float getMonto(Pasaje pasaje) {
        Tarifa tarifa = pasaje.getViaje().getTarifa(pasaje.getAsiento().getTipoDeCoche());
        return tarifa.getMonto() - tarifa.getMonto() * getPorcentajeDeDescuento();
    }
    protected abstract float getPorcentajeDeDescuento();
}
public class Extranjero extends TipoDePasajero {
    @Override
    public float getMonto(Pasaje pasaje) {
        float monto = super.getMonto(pasaje);
```

```
        int cantidadDePasajesVendidosAExtranjeros = pasaje.getViaje().getCantidadDePasajesVendididos(this);
        monto = monto - monto * calcularPorcentajeDeDescuento(cantidadDePasajesVendidosAExtranjeros);
        return monto;
    }
    @Override
    protected float getPorcentajeDeDescuento() {
        // Retorna cero, porque no tiene información suficiente para decidir el % de descuento.
        return 0f;
    }
    private float calcularPorcentajeDeDescuento(int cantidadDePasajesVendidosAExtranjeros) {
        float descuento = .1f;
        if (cantidadDePasajesVendidosAExtranjeros > 20) {
            descuento = 0f;
        }
        return descuento;
    }
}
public class ConPorcentajeDescuentoFijo extends TipoDePasajero {
            private float porcentajeDescuento;
            public ConDescuentoFijo(float porcentajeDescuento){
                        this.porcentajeDescuento=porcentajeDescuento;
            }

            @Override
    protected float getPorcentajeDeDescuento() {
        return porcentajeDescuento;
    }
}
```

B)